

Data Spaces and Federated Data Engineering for Orbital and Space Observations

Laboratory 5: Federated Access Layers with REST, DuckDB over HTTP and GraphQL

Leszek Siwik
Faculty of Space Technologies
AGH University of Krakow

1 Introduction

In previous laboratories, you worked mainly with local files and direct data processing. That approach was useful for understanding:

- basic data manipulation,
- local dataset structure,
- query logic,
- elementary data integration.

However, modern data spaces and data mesh environments are not based on a single access model.

In practice, the same data may be accessed through multiple layers:

- as a service exposed by a provider,
- as a remotely published file,
- as a federated analytical source,
- as part of a higher-level semantic gateway.

This means that a modern consumer may need to work with:

- REST APIs,
- metadata registries,
- data contracts,
- SQL over remote resources,
- GraphQL gateways,
- partial or degraded federation when providers fail.

In this laboratory, you will build one complete multi-layer federation scenario.

You will start with provider APIs, then move to:

- federated REST access,
- metadata-driven provider discovery,
- contract validation,
- DuckDB queries over HTTP-published CSV files,
- a GraphQL gateway exposing unified object-centric access,
- comparison of different federation layers.

The goal of this laboratory is not only to query data, but to understand that federation can happen at different architectural levels.

During this laboratory, you will learn how to:

- expose provider data through HTTP APIs,
- publish data both as JSON and as remote CSV,
- use a provider registry instead of hardcoded endpoints,
- validate provider outputs against a data contract,
- query provider data through REST,
- query provider data through DuckDB over HTTP,
- expose integrated access through GraphQL,
- compare provider-centric, SQL-centric, and object-centric federation styles,
- handle unavailable providers and incomplete results,
- generate a structured federated access report.

2 Required Workspace Structure

Use the following workspace structure:

```
oods-system/
  providers/
    satellite_A/
      api.py
      observations.csv
    satellite_B/
      api.py
      observations.csv
    ground_station/
      api.py
      observations.csv
  gateway/
    graphql_gateway.py
  federation/
    test_single_provider.py
    federated_rest_query.py
    federated_duckdb_query.sql
    layer_comparison.py
    federated_report.py
  contracts/
    observation_schema.json
    providers_registry.json
  reports/
```

The `observations.csv` files may reuse the same or similar datasets as in previous laboratories.

3 Architecture of This Laboratory

This laboratory intentionally uses the same data through multiple access layers.

The architecture is as follows:

- each provider exposes local observations through a REST API,
- the same provider also publishes the raw CSV file through HTTP,
- a federated REST client integrates JSON responses from all providers,
- DuckDB integrates the same providers through remote HTTP CSV access,
- a GraphQL gateway provides one unified semantic access layer,
- a final report compares the results and limitations of all access modes.

This allows you to observe that the same logical data space may be implemented and consumed in different ways.

4 Task 1: Installing Required Tools

This laboratory uses several Python packages and one analytical SQL engine.

Make sure the following tools are available:

- Python 3,
- pip,
- fastapi,
- uvicorn,
- pandas,
- requests,
- duckdb,
- strawberry-graphql[fastapi].

Install the required packages using:

```
pip install fastapi uvicorn pandas requests duckdb "strawberry-graphql[fastapi]"
```

Verify Python and DuckDB availability:

```
python3 --version
duckdb --version
```

Task

Verify that:

- Python is installed,
- DuckDB is available,
- all required Python packages are installed,
- your environment is ready for REST, SQL and GraphQL work.

5 Task 2: Creating Provider APIs

In this laboratory, each provider will expose its local dataset through a small HTTP API.

An API (Application Programming Interface) is a program interface through which another program can request data or services.

In this case:

- each provider will run as a small web service,
- the service will read a local CSV file,
- the service will make the data available through HTTP endpoints,
- other components will later use these endpoints for federation.

The API will be implemented using FastAPI.

FastAPI is a Python framework for creating web APIs. It allows defining endpoints such as:

- / – provider status,
- /observations – all observations,
- /observations/{object_id} – observations for one object,
- /observations.csv – raw CSV file published through HTTP.

Step 1: Create the first provider API

Create the file:

```
providers/satellite_A/api.py
```

with the following content:

```
from fastapi import FastAPI
from fastapi.responses import FileResponse
import pandas as pd

app = FastAPI()

df = pd.read_csv("observations.csv")

@app.get("/")
def root():
    return {"status": "provider online", "provider": "satellite_A"}

@app.get("/observations")
def get_observations():
    return df.to_dict(orient="records")

@app.get("/observations/{object_id}")
def get_object_observations(object_id: str):
    filtered = df[df["object_id"] == object_id]
    return filtered.to_dict(orient="records")

@app.get("/observations.csv")
def get_csv():
    return FileResponse("observations.csv", media_type="text/csv")
```

What this script does

The script above performs the following actions:

- `from fastapi import FastAPI`
creates a web API application,
- `from fastapi.responses import FileResponse`
allows returning a file directly through HTTP,
- `import pandas as pd`
imports the Pandas library used to read CSV data,
- `app = FastAPI()`
creates the API application object,
- `df = pd.read_csv("observations.csv")`
loads the local dataset into memory,
- `@app.get("/")`
defines the root endpoint returning a simple provider status message,
- `@app.get("/observations")`
defines an endpoint returning the full dataset as JSON,
- `@app.get("/observations/{object_id}")`
defines an endpoint returning only observations matching the selected object identifier,
- `@app.get("/observations.csv")`
defines an endpoint that publishes the raw CSV file through HTTP.

Notice that the same provider exposes the same data in two forms:

- as structured JSON for API-based access,
- as a CSV file for remote analytical querying.

This is important because later in the laboratory different federation layers will consume the same provider in different ways.

Step 2: Create the remaining providers

Create analogous files for:

- providers/satellite_B/api.py,
- providers/ground_station/api.py.

Do not rewrite the whole file from scratch. First copy the working version from `satellite_A`, then adjust only the provider-specific part returned by the root endpoint.

For example, in `satellite_B/api.py`:

```
return {"status": "provider online", "provider": "satellite_A"}
```

should become:

```
return {"status": "provider online", "provider": "satellite_B"}
```

and analogously for `ground_station`.

At the end of this part you should have the API for all providers. Each API must support:

- /,
- /observations,
- /observations/{object_id},
- /observations.csv.

and in the directory for each provider(s) you should get:

```
api.py  
observations.csv
```

6 Task 3: Running Provider Services

Now, let's start all providers in separate terminals:

```
cd providers/satellite_A  
uvicorn api:app --port 8001
```

```
cd providers/satellite_B  
uvicorn api:app --port 8002
```

```
cd providers/ground_station  
uvicorn api:app --port 8003
```

A correct startup should display output similar to:

```
INFO:      Uvicorn running on http://127.0.0.1:8001
```

You may test providers in a browser:

- <http://127.0.0.1:8001/>
- <http://127.0.0.1:8001/observations>
- <http://127.0.0.1:8001/docs>

Verify that:

- the root endpoint returns provider status,
- the observations endpoint returns JSON data,
- the CSV endpoint is accessible through HTTP,
- the API documentation is available under `/docs`.

7 Task 4: Querying a Single Provider via REST

In the previous tasks, you created and started provider APIs.

Now we will use it, i.e. instead of opening data in files, you will request it from a provider using HTTP.

This is the basic mechanism behind REST-based data access.

When you execute a request such as:

```
http://127.0.0.1:8001/observations
```

you are:

- sending an HTTP request to a running provider,
- the provider processes the request,
- the provider returns data in JSON format,
- your script receives and processes that data.

To perform HTTP requests in Python, we use the `requests` library.

Step 1: Create a test script

Create the file:

```
federation/test_single_provider.py
```

with the following content:

```
import requests

response = requests.get("http://127.0.0.1:8001/observations")
data = response.json()

print("NUMBER OF OBSERVATIONS:", len(data))
print("FIRST RECORD:")
print(data[0])
```

What this script does

- `import requests`
loads the HTTP client library,
- `requests.get(...)`
sends an HTTP GET request to the provider,
- `response`
contains the full HTTP response returned by the provider,
- `response.json()`
converts the response body from JSON into Python data structures (list of dictionaries),
- `len(data)`
counts how many records were returned,
- `data[0]`
prints the first observation from the dataset.

Important: the provider returns data in JSON format, but after calling `.json()`, you are working with standard Python objects.

Step 2: Query a selected object

Extend the script or create a second one:

```
import requests

object_id = "OBJ-003"
response = requests.get(f"http://127.0.0.1:8001/observations/{object_id}")
data = response.json()

print("OBJECT:", object_id)
print("RESULTS:", len(data))
print(data)
```

What changes here

- the URL now includes `/observations/OBJ-003`,
- the provider filters the dataset internally,
- only matching records are returned,
- the client does not need to filter data manually.

To run those script(s), make sure:

- the provider is running (Task 3),
- you are in the project root directory (e.g. `oods-system/`).

Run:

```
python3 federation/test_single_provider.py
```

Common problems

- **Connection error**
The provider is not running or wrong port is used.
- **Empty result**
The object does not exist in this provider.
- **JSON decode error**
The endpoint URL is incorrect (e.g. typo in path).

Task

Create one script that:

- queries `satellite_A` for all observations,
- queries both `satellite_A` and `satellite_B` for a selected object identifier,
- prints the total number of observations in `satellite_A`,
- prints how many observations of the selected object were returned by each provider,
- compares the results in one short summary.

And you should get something similar to:

```
satellite_A:
NUMBER OF OBSERVATIONS: 6

OBJ-003 RESULTS: 1

satellite_B:
OBJ-003 RESULTS: 1

COMPARISON:
OBJ-003 is present in both providers.
```

8 Task 5: Provider Registry and Data Contract

Until now, provider addresses were written directly in Python scripts.

For example:

```
("satellite_A", "http://127.0.0.1:8001")
```

This approach works for small experiments, but it becomes problematic when:

- providers change addresses,
- new providers are added,
- some providers are removed,
- multiple clients need to use the same configuration.

In real data spaces, provider definitions are not hardcoded. Instead, they are stored in metadata structures such as registries.

At the same time, we also need a way to define what kind of data providers are expected to expose.

This is done using a **data contract**. The first step is defining the provider registry which describe:

- which providers exist,
- how they are identified,
- where they are accessible.

Create:

```
contracts/providers_registry.json
```

with the following content:

```
[
  {"name": "satellite_A", "url": "http://127.0.0.1:8001"},
  {"name": "satellite_B", "url": "http://127.0.0.1:8002"},
  {"name": "ground_station", "url": "http://127.0.0.1:8003"}
]
```

This file becomes the single source of truth for all providers.

Instead of modifying code, we can now change the system behavior by editing this file.

The second step is defining the data contract which defines what structure each provider must follow.

Create:

```
contracts/observation_schema.json
```

with the following content:

```
{
  "required_fields": [
    "timestamp",
    "object_id",
    "temperature",
    "velocity"
  ]
}
```

This means that every observation returned by a provider is expected to contain these fields.

These two components introduce key ideas used in real data spaces:

- **Provider registry**
decouples federation logic from hardcoded configuration,

- **Data contract**

defines expectations and enables validation of provider outputs.

Later in the laboratory, you will:

- load providers dynamically from the registry,
- validate whether providers follow the contract,
- detect missing or inconsistent data.

To sum up, at the end of this step you should have the directory:

```
contracts/
```

containing:

```
providers_registry.json
observation_schema.json
```

9 Task 6: Building a Federated REST Client

In the previous task, you created a provider registry.

Now we will use it to build our first federated client.

A federated client is a consumer that:

- knows about multiple providers,
- queries them one by one,
- retrieves their data,
- integrates all returned results into one combined view.

This task is the first step from single-provider access toward true data federation.

Step 1: Create the federated client

Create:

```
federation/federated_rest_query.py
```

with a structure similar to the following:

```
import json
import requests

with open("../contracts/providers_registry.json", "r") as f:
    providers = json.load(f)

all_results = []

for provider in providers:
    provider_name = provider["name"]
    provider_url = provider["url"]

    response = requests.get(f"{provider_url}/observations")
    data = response.json()

    for row in data:
        row["provider"] = provider_name
        all_results.append(row)

print("TOTAL RECORDS:", len(all_results))
for row in all_results:
```

```
print(row)
```

What this script does

- `import json`
loads the module used to read JSON configuration files,
- `import requests`
loads the library used to send HTTP requests,
- `with open(...)`
opens the provider registry file,
- `json.load(f)`
converts the registry from JSON into Python objects,
- `providers`
contains a list of provider definitions (name + URL),
- `all_results = []`
creates an empty list for storing integrated observations,
- `for provider in providers:`
loops over all providers listed in the registry,
- `requests.get(...)`
sends an HTTP request to each provider,
- `response.json()`
converts the response into Python data structures,
- `row["provider"] = provider_name`
adds information about the source provider,
- `all_results.append(row)`
adds the observation to the combined dataset.

Pls, pay attention, that without:

```
row["provider"] = provider_name
```

the final dataset would contain observations, but it would not be possible to determine their origin.

After adding provider identity:

- the dataset becomes traceable,
- observations can be grouped by provider,
- inconsistencies between providers can be detected,
- the federation result becomes interpretable.

As the result, even though providers are independent:

- each one runs separately,
- each one stores its own data,
- each one answers its own requests,

the federated client merges all results into one list.

This means that multiple distributed datasets are treated as a single integrated dataset from the perspective of the consumer.

How to run the script

Before running the client, make sure that:

- all providers are running,
- `providers_registry.json` exists,

- you are in the main project directory (e.g. oods-system/).

Run:

```
python3 federation/federated_rest_query.py
```

Common problems

- **File not found**
The registry file path is incorrect or the script is run from a wrong directory.
- **Connection error**
One or more providers are not running.
- **Unexpected result count**
Providers may contain different numbers of observations.

At this stage, you are no longer querying a single provider.

Instead, you are:

- querying multiple independent providers,
- integrating their responses,
- constructing a unified data view.

This is the core idea of federation in data spaces.

Task

Using the example above as a starting point, extend the federated client so that it:

- loads providers from the registry,
- retrieves observations from all providers,
- combines all returned observations into one logical dataset,
- computes how many observations come from each provider,
- identifies which provider contributes the largest number of observations,
- prints a short summary of the federated result.

Your summary should answer questions such as:

- how many observations are available in total?
- how are they distributed across providers?
- which provider contributes the most data?

Expected Output

```
TOTAL RECORDS: 15

PER-PROVIDER COUNTS:
satellite_A: 6
satellite_B: 5
ground_station: 4

LARGEST PROVIDER:
satellite_A
```

10 Task 7: Object-Centric REST Federation and Completeness

In observational systems, analysis is often object-centric rather than file-centric.

This means that instead of retrieving all available data from a single provider, we are interested in collecting all observations of a specific object across the entire data space.

This type of query is very common in space systems: we want to reconstruct the full observation history of a satellite, debris object, or atmospheric event using multiple independent sources.

However, this introduces a key challenge: even if a query returns results, it does not guarantee completeness. Some providers may not contain the object, or may fail to respond.

In this task, you will move from simple federation to reasoning about completeness of federated results.

Extend the federated client so that it queries:

```
/observations/OBJ-003
```

for every provider.

A possible implementation is:

```
import json
import requests

with open("../contracts/providers_registry.json", "r") as f:
    providers = json.load(f)

object_id = "OBJ-003"
results = []
providers_with_object = []

for provider in providers:
    provider_name = provider["name"]
    provider_url = provider["url"]

    response = requests.get(f"{provider_url}/observations/{object_id}")
    data = response.json()

    if data:
        providers_with_object.append(provider_name)

    for row in data:
        row["provider"] = provider_name
        results.append(row)

print("OBJECT:", object_id)
print("TOTAL OBSERVATIONS:", len(results))
print("PROVIDERS CONTAINING OBJECT:")
for name in providers_with_object:
    print(name)
```

This script introduces object-centric federation:

- instead of retrieving all observations, we query each provider for a specific object,
- results are collected across all providers,
- we track which providers contain the object,
- we begin to reason about completeness of the federated result.

Task

Using the example above as a starting point, extend the script so that it:

- retrieves all observations of OBJ-003 across all providers,
- identifies which providers contain this object,

- compares this list with the full provider registry,
- determines which providers are missing the object,
- evaluates whether the federated result is complete.

In your implementation, explicitly distinguish between:

- providers that returned data,
- providers that returned no data,
- providers that were expected but are missing from the result.

Expected Output

```
OBJECT: OBJ-003
TOTAL OBSERVATIONS: 3

PROVIDERS CONTAINING OBJECT:
satellite_A
satellite_B
ground_station

MISSING PROVIDERS:
none

COMPLETENESS: YES
```

11 Task 8: Contract Validation and Provider Availability

A federated system should not assume that every provider is:

- online,
- responsive,
- contract-compliant.

This means that before trusting the result of a federated query, the client should verify two things:

- **availability** – can the provider be reached?
- **contract compliance** – does the provider return the expected data structure?

In this task, you will create a validation script that checks both conditions.

Step 1: Create the validation script

Create:

```
federation/provider_validation.py
```

with a structure similar to the following:

```
import json
import requests

with open("../contracts/providers_registry.json", "r") as f:
    providers = json.load(f)

with open("../contracts/observation_schema.json", "r") as f:
    contract = json.load(f)

required_fields = contract["required_fields"]
```

```

for provider in providers:
    provider_name = provider["name"]
    provider_url = provider["url"]

    try:
        response = requests.get(f"{provider_url}/observations", timeout=2)
        data = response.json()

        if len(data) == 0:
            print(provider_name, ": EMPTY DATASET")
            continue

        missing = [field for field in required_fields if field not in data[0]]

        if missing:
            print(provider_name, ": VIOLATION")
            print("Missing fields:", missing)
        else:
            print(provider_name, ": OK")

    except Exception:
        print(provider_name, ": UNAVAILABLE")

```

This script:

- it loads the provider list from the registry,
- it loads the required field list from the contract,
- it sends a request to each provider,
- it checks whether the provider responds within the timeout,
- it checks whether the returned data contains the expected fields,
- it classifies the provider status.

What the possible statuses mean

- OK
The provider is reachable and returns data matching the expected contract.
- VIOLATION
The provider is reachable, but the returned data does not match the contract.
- UNAVAILABLE
The provider could not be reached or did not respond in time.
- EMPTY DATASET
The provider responded correctly, but returned no observations.

Why timeout is important

Without a timeout, a client may wait too long for a slow or unavailable provider.

The instruction:

```
requests.get(..., timeout=2)
```

means that the client will wait at most 2 seconds for a response.

This is important in federated environments because:

- one slow provider should not block the whole federation,
- degraded providers must be detected explicitly,
- incomplete results should be recognized as incomplete.

To run the script:

Make sure that:

- the provider APIs are running,
- the files `providers_registry.json` and `observation_schema.json` exist,
- you are in the main project directory.

Run:

```
python3 federation/provider_validation.py
```

Task

Using the example above as a starting point, extend the validation script so that it:

- classifies every provider as OK, VIOLATION, UNAVAILABLE, or EMPTY DATASET,
- counts how many providers fall into each category,
- prints the list of problematic providers,
- evaluates whether the current federation state is fully reliable or degraded.

Treat the federation as:

- RELIABLE – if all providers are OK,
- DEGRADED – if at least one provider is VIOLATION, UNAVAILABLE, or EMPTY DATASET.

Expected Output

```
satellite_A: OK
satellite_B: OK
ground_station: UNAVAILABLE

SUMMARY:
OK: 2
VIOLATION: 0
UNAVAILABLE: 1
EMPTY DATASET: 0

PROBLEMATIC PROVIDERS:
ground_station

FEDERATION STATE: DEGRADED
```

12 Task 9: Querying Provider Data with DuckDB over HTTP

The same providers can now be consumed not only as REST services, but also as remote analytical data sources.

Each provider already exposes:

```
/observations.csv
```

This means that DuckDB can query provider datasets directly over HTTP, without manually downloading the files.

This introduces a different access model:

- REST : record-by-record access (application-level),
- DuckDB : SQL-based analytical access (query-level).

Step 1: Start DuckDB

Run:

```
duckdb
```

If direct remote HTTP reading does not work in your environment, load the required extension:

```
INSTALL httpfs;  
LOAD httpfs;
```

Step 2: Query a Remote Provider

Test a remote query:

```
SELECT * FROM 'http://127.0.0.1:8001/observations.csv';
```

Inspect the schema:

```
DESCRIBE SELECT * FROM 'http://127.0.0.1:8001/observations.csv';
```

What is happening here

- DuckDB sends an HTTP request to the provider,
- the CSV file is read directly from the remote service,
- no manual download is required,
- the dataset is treated as a SQL table.

Task

Using DuckDB, compare two remote providers accessed through HTTP.

Your analysis should:

- count how many observations are available in `satellite_A`,
- count how many observations are available in `satellite_B`,
- count how many observations of `OBJ-003` are available in each provider,
- compare the schemas of both remote datasets,
- determine whether the two providers appear structurally compatible.

Expected Output

```
satellite_A:  
TOTAL OBSERVATIONS: 6  
OBJ-003 OBSERVATIONS: 1  
  
satellite_B:  
TOTAL OBSERVATIONS: 5  
OBJ-003 OBSERVATIONS: 1  
  
SCHEMA COMPARISON:  
satellite_A and satellite_B are compatible
```


13 Task 10: Federated DuckDB Query over HTTP

DuckDB can integrate multiple remote providers using SQL.

This enables a different type of federation: instead of manually combining results in Python, we can express the entire federation as a single SQL query.

Create:

```
federation/federated_duckdb_query.sql
```

with a structure similar to the following:

```
SELECT 'satellite_A' AS provider, *
FROM 'http://127.0.0.1:8001/observations.csv'
UNION ALL
SELECT 'satellite_B' AS provider, *
FROM 'http://127.0.0.1:8002/observations.csv'
UNION ALL
SELECT 'ground_station' AS provider, *
FROM 'http://127.0.0.1:8003/observations.csv';
```

You may also query object-centric results:

```
SELECT provider, timestamp, object_id, temperature, velocity
FROM (
  SELECT 'satellite_A' AS provider, *
  FROM 'http://127.0.0.1:8001/observations.csv'
  UNION ALL
  SELECT 'satellite_B' AS provider, *
  FROM 'http://127.0.0.1:8002/observations.csv'
  UNION ALL
  SELECT 'ground_station' AS provider, *
  FROM 'http://127.0.0.1:8003/observations.csv'
)
WHERE object_id = 'OBJ-003';
```

What is happening here

- multiple remote providers are combined using `UNION ALL`,
- each dataset is labeled with its provider,
- the result behaves like a single logical table,
- filtering and aggregation can be performed using SQL.

Task

Using DuckDB, perform a federated analysis of all providers.

Your analysis should:

- compute the total number of observations across all providers,
- compute how many observations come from each provider,
- retrieve all observations of `OBJ-003`,
- identify which provider contributes the most data,
- verify whether all providers are represented in the result.

Expected Output

```
TOTAL OBSERVATIONS: 15
OBJ-003 OBSERVATIONS: 3
```

```
satellite_A: 6
satellite_B: 5
ground_station: 4
```

```
LARGEST PROVIDER:
satellite_A
```

Interpretation

Compare this SQL-based federation with the REST-based approach used earlier:

- Which approach requires less code on the client side?
- Which approach is easier for aggregation and counting?
- Which approach gives a clearer global view of the data space?
- Does SQL federation make completeness easier to assess?

14 Task 11: GraphQL Gateway for Unified Access

REST and SQL provide two useful access layers, but a data space may also expose a semantic, object-centric access layer.

GraphQL is a query language for APIs that allows clients to request exactly the data they need from a single endpoint.

Instead of calling multiple REST endpoints manually, a client sends one query to the GraphQL gateway, which internally retrieves and combines data from multiple providers.

In this laboratory, the GraphQL gateway acts as a unified access layer on top of the federated providers.

Create:

```
gateway/graphql_gateway.py
```

with a structure similar to the following:

```
import json
import requests
import strawberry
from fastapi import FastAPI
from strawberry.fastapi import GraphQLRouter

with open("../contracts/providers_registry.json", "r") as f:
    PROVIDERS = json.load(f)

@strawberry.type
class Observation:
    provider: str
    timestamp: str
    object_id: str
    temperature: float
    velocity: float

def fetch_all():
    results = []
    for provider in PROVIDERS:
```

```

        provider_name = provider["name"]
        provider_url = provider["url"]

        response = requests.get(f"{provider_url}/observations", timeout=2)
        data = response.json()

        for row in data:
            results.append(Observation(
                provider=provider_name,
                timestamp=str(row["timestamp"]),
                object_id=str(row["object_id"]),
                temperature=float(row["temperature"]),
                velocity=float(row["velocity"])
            ))
        return results

def fetch_object(object_id: str):
    results = []
    for provider in PROVIDERS:
        provider_name = provider["name"]
        provider_url = provider["url"]

        response = requests.get(f"{provider_url}/observations/{object_id}", timeout=2)
        data = response.json()

        for row in data:
            results.append(Observation(
                provider=provider_name,
                timestamp=str(row["timestamp"]),
                object_id=str(row["object_id"]),
                temperature=float(row["temperature"]),
                velocity=float(row["velocity"])
            ))
        return results

@strawberry.type
class Query:
    @strawberry.field
    def observations(self) -> list[Observation]:
        return fetch_all()

    @strawberry.field
    def observations_by_object(self, object_id: str) -> list[Observation]:
        return fetch_object(object_id)

schema = strawberry.Schema(query=Query)

app = FastAPI()
app.include_router(GraphQLRouter(schema), prefix="/graphql")

```

What is happening here

- the gateway loads providers from the registry,
- it internally queries all providers using REST,
- it converts raw JSON data into structured objects,
- it exposes a single unified API through GraphQL,

- clients no longer interact with providers directly.

Running the gateway

Run:

```
cd gateway
uvicorn graphql_gateway:app --port 9000
```

If the server starts correctly, it should display:

```
Uvicorn running on http://127.0.0.1:9000
```

How to use the GraphQL gateway

Open your browser and go to:

```
http://127.0.0.1:9000/graphql
```

You should see an interactive GraphQL interface.

You can execute queries such as:

```
{
  observations {
    provider
    objectId
    timestamp
  }
}
```

or:

```
{
  observationsByObject(objectId: "OBJ-003") {
    provider
    timestamp
    temperature
  }
}
```

Task

Create and run the GraphQL gateway, and verify that it works correctly.

Then:

- execute a query that retrieves all observations,
- execute a query that retrieves only selected fields (e.g., provider and timestamp),
- execute a query for OBJ-003,
- compare the results with the REST and DuckDB outputs.

Expected Output

```
OBJ-003 observations:
- satellite_A ...
- satellite_B ...
- ground_station ...
```

15 Task 12: Comparing Access Layers

At this stage you have three access styles:

- provider-level REST access,
- DuckDB SQL federation over HTTP,
- GraphQL gateway federation.

Each style has different strengths.

REST:

- simple and explicit,
- close to provider ownership,
- useful for service-level consumption.

DuckDB over HTTP:

- analytical and SQL-oriented,
- useful for aggregation and counting,
- convenient for querying published files remotely.

GraphQL:

- object-centric,
- semantically unified,
- allows clients to request only the fields they need.

Task

Create:

```
reports/access_layer_comparison.txt
```

The file should answer:

- which access style is easiest to implement,
- which is best for SQL-style analytical processing,
- which is best for object-centric access,
- which introduces the highest architectural complexity,
- which is closest to a data space or data mesh access layer.

16 Task 14: Final Federated Access Report

Create:

```
federation/federated_report.py
```

The script should:

- load provider metadata from the registry,
- check provider availability,
- validate provider outputs against the contract,
- retrieve all observations through REST federation,
- perform object-level analysis for a selected object,
- compare REST, DuckDB and GraphQL results,
- record incomplete or degraded federation,
- generate a structured report file.

The report should be saved as:

```
reports/federated_access_report_YYYYMMDD_HHMMSS.txt
```

Report Requirements

The report must include:

- list of registered providers,
- provider availability status,
- contract validation status,
- total number of observations,
- total number of distinct objects,
- total observations for a selected object,
- providers containing the selected object,
- result of REST federation,
- result of DuckDB federation,
- result of GraphQL federation,
- a short note on completeness and failures.

Expected Output

```
FEDERATED ACCESS REPORT
-----
Generated at: 2026-04-20 10:30:00

[REGISTERED PROVIDERS]
- satellite_A
- satellite_B
- ground_station

[PROVIDER STATUS]
satellite_A: AVAILABLE
satellite_B: AVAILABLE
ground_station: AVAILABLE

[CONTRACT VALIDATION]
satellite_A: OK
satellite_B: OK
ground_station: OK

[GLOBAL STATISTICS]
Total observations: 15
Distinct objects: 6

[OBJECT ANALYSIS: OBJ-003]
Providers containing object:
- satellite_A
- satellite_B
- ground_station
Total observations: 3

[ACCESS LAYERS]
REST RESULT: 15
DUCKDB RESULT: 15
GRAPHQL RESULT: 15

[COMPLETENESS]
Federation complete: YES
```

17 Submission

Submit your work using both a version control repository and an archive uploaded to the UPEL platform.

Both submissions are required for the laboratory to be considered complete.

Repository Submission

Your repository must include:

- all provider API files in the `providers` directory,
- the GraphQL gateway in the `gateway` directory,
- all federation scripts in the `federation` directory,
- all contract and registry files in the `contracts` directory,
- generated report files in the `reports` directory.

After completing the laboratory:

- commit your changes,
- push them to your repository,
- create a **Merge Request** (GitLab) or **Pull Request** (GitHub).

The request must clearly indicate that it contains the solution for:

Laboratory 5: Federated Access Layers with REST, DuckDB over HTTP and GraphQL

UPEL Submission

Upload a compressed archive containing the complete project workspace:

oods-system/
 providers/
 gateway/
 federation/
 contracts/
 reports/

The archive must be named:

lab05_oods_<surname>.tar.gz

Report File Requirement

The submitted report must contain:

- provider registry information,
- provider availability status,
- contract validation results,
- global observation statistics,
- object-level analysis,
- comparison of REST, DuckDB and GraphQL federation.

Final Note

This laboratory is intentionally broader than previous ones.

Its purpose is to show that a modern data space is not defined by a single access method.

Instead, the same provider data may be consumed:

- as a service,
- as a published remote file,
- as a federated SQL source,
- as a semantic GraphQL layer.

Understanding these layers is an important step toward working with realistic data spaces, data mesh environments, and external metadata-driven systems in later laboratories.